TRD: Customer Churn Prediction

Oct 12, 2026 · @Vedant

Technical overview

This TRD specifies how to build the churn prediction pipeline described in the PRD: a batch, notebook-and-module Python project that turns a customer usage file into a trained classifier, an evaluation report and a list of churn drivers.

Design principles

- Reproducible: fixed seeds, pinned dependencies, one command to rerun.
- Leakage-safe: every transform is fitted on training data only, inside a scikit-learn Pipeline.
- Simple first: a logistic regression baseline before any tree ensemble.
- Explainable: every model ships with importance and SHAP outputs.

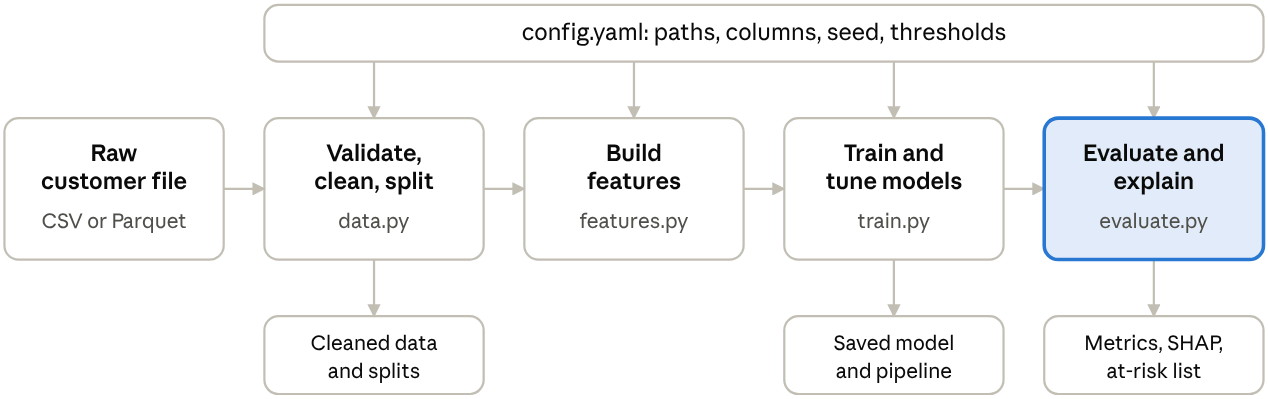
Constraints

- Runs on a laptop or free notebook environment; no GPU needed for tabular data.
- Batch scoring only; no serving infrastructure in v1.
- Dataset size and schema are not yet fixed, so the code reads column names from a config file rather than hard-coding them.

System architecture

The system is a linear batch pipeline of five stages, each in its own module, all reading settings from one config file.

One config file drives five stages from raw data to recommendations



pipeline architecture · 5 stages, 3 outputs

Stages pass data forward in order; the cleaned splits, the saved model with its preprocessing pipeline, and the evaluation outputs are the artifacts written to disk.

Tech stack and environment

The stack is standard Python data science tooling, chosen for familiarity and low setup cost.

Layer	Choice	Purpose
Language	Python 3.10 or newer	Core implementation
Data handling	pandas, NumPy	Loading, cleaning, feature tables
Modeling	scikit-learn, XGBoost or LightGBM	Pipelines, baselines, gradient boosting
Imbalance	scikit-learn class weights, imbalanced-learn (optional)	Handling minority churn class
Explainability	SHAP	Per-feature contribution values
Visualisation	matplotlib, seaborn	EDA and evaluation charts
Persistence	joblib	Saving model and preprocessing pipeline

Layer	Choice	Purpose
Config	YAML file	Column names, paths, seeds, thresholds
Environment	venv or conda with requirements.txt	Pinned, reproducible install
Version control	Git	Code and notebook history

Exact library versions are pinned in requirements.txt when the environment is first created.

Data pipeline design

The data layer loads the raw file once, validates it, and writes a cleaned table plus fixed train, validation and test splits.

Ingestion. `load_data(path)` reads CSV or Parquet, enforces expected dtypes from the config, and drops the identifier column from the modelling frame while keeping it for the final at-risk list.

Validation checks (fail loudly)

- Required columns exist and the target is strictly 0 or 1.
- No duplicate customer identifiers.
- No negative tenure, usage or charges.
- Missing-value share per column is reported; columns above a configured threshold are flagged for review.

Cleaning rules

- Numeric missing values: median imputation, fitted on training data.
- Categorical missing values: a separate "missing" category.
- Numeric columns stored as text (for example blank strings in a charges column) are coerced and the failures counted.

Splits. Stratified 70/15/15 train, validation and test split with a fixed seed. The test set is touched once, at final evaluation. Cross-validation runs on the training set only.

Leakage guard. Any column recorded at or after the churn event is listed in the config as excluded; a test asserts none appear in the feature matrix.

Feature engineering specification

Features are built in `features.py` as a scikit-learn ColumnTransformer so the identical steps run in training and scoring. Field names below are placeholders until the real schema is known.

Feature	Definition	Source fields
tenure_bucket	Tenure binned (for example 0-6, 7-12, 13-24, 25+ months)	tenure
usage_frequency	Sessions or logins per active month	sessions, tenure
usage_trend	Recent usage divided by earlier usage, or last-period change	usage by period
tickets_per_month	Support tickets divided by tenure in months	tickets, tenure
avg_resolution_time	Mean time to resolve tickets	ticket timestamps
charge_per_usage	Monthly charges divided by usage volume	charges, usage
contract_type, plan, payment_method	One-hot encoded categoricals	account fields
numeric scaling	StandardScaler for logistic regression only; trees use raw values	numeric features

Rules

- Divisions guard against zero with a small constant or a null flag.
- Highly correlated pairs (above a configured correlation) are reviewed and one is dropped for the linear model.
- Each feature has a one-line docstring stating its business meaning, which the final report reuses.

Model design and training

Each model is wrapped in one Pipeline of preprocessing plus estimator, so cross-validation refits every transform inside each fold.

Model	Role	Key settings to tune
Dummy classifier	Floor to beat	Most-frequent strategy
Logistic regression	Interpretable baseline	Regularisation strength, class_weight
Random forest	Non-linear benchmark	Trees, max depth, min samples per leaf, class_weight
XGBoost or LightGBM	Expected best performer	Learning rate, depth, trees, scale_pos_weight

Training procedure

1. Fit all models with default settings and record validation ROC-AUC and PR-AUC.
2. Tune the top two with RandomizedSearchCV: stratified 5-fold, scored on PR-AUC, fixed seed.
3. Pick the final model on validation scores, breaking ties in favour of the simpler model.
4. Refit the winner on train plus validation data and save it with joblib, with the config and seed stored beside it.

Imbalance handling. Class weights are the default. Oversampling such as SMOTE is tried only inside the cross-validation pipeline, never before the split.

Evaluation and explainability implementation

`evaluate.py` scores the saved model on the untouched test set exactly once and writes every figure to the reports folder.

Output	Method
ROC-AUC, PR-AUC	sklearn.metrics on test probabilities
Confusion matrix, precision, recall, F1	At the chosen threshold

Output	Method
Threshold selection	Sweep thresholds on validation probabilities; pick the highest precision that still meets the recall target
Lift and cumulative gains	Customers sorted by predicted probability, grouped into deciles
Calibration curve	Checks whether predicted probabilities match observed churn rates
Global importance	Model feature importance and permutation importance on validation data
Explanations	SHAP summary plot and dependence plots for the top drivers
At-risk list	Customer id, churn probability and top three contributing features

Turning drivers into recommendations. For each of the top drivers, the report pairs the SHAP direction with the observed churn rate by segment, then states one suggested action and its evidence. A short template keeps every recommendation in the same format: driver, evidence, suggested action, caveat that this is an association.

Code structure and interfaces

Notebooks hold exploration and narrative; reusable logic lives in `src/` so notebooks stay short and the pipeline can run from the command line.

Module	Key functions	Responsibility
config.yaml	n/a	Paths, column names, excluded columns, seed, target recall
src/data.py	load_data, validate, split	Ingestion, checks, stratified splits
src/features.py	build_preprocessor	ColumnTransformer with all engineered features
src/train.py	train_models, tune, save_model	Baselines, search, final fit and persistence

Module	Key functions	Responsibility
src/evaluate.py	evaluate, choose_threshold, explain	Metrics, lift, SHAP, at-risk list
run_pipeline.py	main	Runs all stages end to end from the config

Reproducibility

- One global seed from the config is passed to splits, models and searches.
- requirements.txt pins versions; the saved model stores the config and library versions beside it.
- Raw data is never modified; processed data is regenerated by the pipeline.
- Git ignores large data files and model binaries; the README states how to obtain them.

Testing and quality

Tests focus on the failures that silently ruin churn models: leakage, broken splits and drifting feature logic.

Test	What it asserts
Schema validation	Missing columns, non-binary target or duplicate ids raise an error
Split integrity	No customer appears in more than one split; churn rate matches across splits
Leakage guard	Excluded post-churn columns are absent from the feature matrix
Feature unit tests	Each engineered feature returns the expected value on a small hand-made frame, including zero-division cases
Pipeline fit	Preprocessing is fitted on training rows only (checked by comparing learned statistics)
Determinism	Two runs with the same seed give identical metrics

Test	What it asserts
Sanity check	A model on shuffled labels scores near chance, confirming no hidden leakage

Tests run with pytest. A model is accepted only when it beats the baseline on validation and meets the success targets in the PRD on the test set.

Technical risks, decisions and open questions

The main technical risk is that the real dataset's schema or size differs from the assumptions here, so the config-driven design absorbs that change.

Decision	Choice	Alternative and why not (for now)
Model family	Logistic regression baseline plus gradient boosting	Deep learning: tabular data of this kind rarely benefits and costs explainability
Imbalance method	Class weights	SMOTE: riskier for leakage, tried only inside folds
Primary tuning metric	PR-AUC	Accuracy: misleading with a minority class
Serving	Batch scoring from saved model	API or dashboard: out of scope for v1
Config	YAML file	Hard-coded names: breaks when the schema changes

Technical risks

- Small datasets make tuning noisy; repeated cross-validation and a simple baseline limit overfitting.
- Time-based churn data may need a time-ordered split instead of a random one; this changes the splitting code only.
- SHAP runtime grows with data size; a sample of rows is used if needed.

Open questions

- [ ] Final dataset and its real column names?

- [ ] Is a random split acceptable, or must the split respect time order?
- [ ] Should the project add a small dashboard or stay as notebooks and a report?
- [ ] Does the team prefer XGBoost or LightGBM?